

PayLedger — Interview Prep Pack

Everything you need to talk about this project for 10–15 minutes with confidence, plus the questions an interviewer will fire at you and exactly how to answer each one.

How to use this: read it twice. Then practice the **10-Minute Script** out loud 3 times. Then drill the **Q&A** section. Don't memorize word-for-word — understand the *why* behind each decision, because that's what gets tested.

☐ THE MASTER TEMPLATE — "Tell Me About Your Project"

This is the structure to follow **every single time** an interviewer says "walk me through your project." Hit these beats in this order. Don't ramble — each beat has a job.

Problem (30s) → Your Role (20s) → System Architecture / high-level flow → Tech Choices → Challenges Faced → Results & Impact → Future Improvements

The **1-minute version** drops the last two beats: *Problem → Role → Architecture → Tech Choices → Challenges → Impact*. Use the 1-min version as your default; expand only if they lean in or ask "go deeper."

☐ THE 1-MINUTE VERSION (say this verbatim until it's natural)

[Problem — 30s]

"The problem I set out to solve is that moving money between accounts is deceptively hard. A real payment system has to guarantee three things even when things go wrong: balances must always be exact and auditable, the same request must never double-charge a customer, and two payments hitting the same account at once can't corrupt the balance. Most portfolio projects are just CRUD — I wanted to build something that actually handles these production-grade failure and concurrency problems."

[Your Role — 20s]

"I built the entire thing solo, end to end — I designed the database schema, built the Spring Boot backend, wrote three separate React frontends for the three user roles, and deployed it all live on Render and Vercel. So every architectural decision in it is mine."

[Architecture — high-level flow]

"At a high level: three React portals — customer, back-office, and admin — talk over REST to a Spring Boot API secured with JWT. The API sits on top of three stores — MySQL as the source of truth, Redis for short-lived data like OTPs and idempotency keys, and Kafka for streaming transaction events out asynchronously."

[Tech Choices]

"Backend is Spring Boot 3 with Java 17, Spring Security with JWT, JPA/Hibernate for data access, TiDB which is MySQL-compatible, Redis on Upstash, Apache Kafka, and Twilio for SMS. Frontends are React 18 with Vite. A few decisions were deliberate — I store all money as integer subunits, not decimals, for exact arithmetic; and I chose optimistic locking over pessimistic for throughput."

[Challenges]

"The three hardest problems were: preventing double-charges on retried requests, which I solved with idempotency keys in Redis; preventing lost-update race conditions on concurrent transfers, solved with optimistic locking using a version column; and making sure a notification failure could never roll back a payment, solved with a Kafka transactional outbox."

[Impact]

"The result is a fully working system deployed live that demonstrates real fintech patterns — double-entry accounting, idempotency, optimistic concurrency control, and event-driven async processing — not just basic CRUD."

Then stop and offer: *"I can go deeper on the architecture or any of those challenges."*

□ THE EXPANDED VERSION — each beat, fleshed out

Use these when they ask for more on a specific beat, or when you have 5+ minutes.

1. Problem (the "why this exists") Payment systems have a zero-tolerance correctness requirement — a lost rupee or a double-charge is a real failure, not a cosmetic bug. The core problems are: (a) **exactness** — money math can't use floating point; (b) **idempotency** — networks retry and users double-click, so the same request must not run twice; (c) **concurrency** — simultaneous transfers on one account must not overwrite each other; (d) **partial failure** — an external call like SMS must never corrupt or slow a payment; and (e) **auditability** — every balance must be reconstructable. PayLedger is built to solve all five.

2. Your Role Solo full-stack developer. Owned the entire lifecycle: data modeling, backend services, security, three frontends, deployment, and the infrastructure choices (TiDB, Upstash Redis, Render, Vercel). No template, no starter — built from scratch.

3. System Architecture (high-level flow) Three independent React apps (customer / back-office / admin), each its own deployment, call a single Spring Boot REST API over HTTPS with a JWT bearer token. A `JwtFilter` authenticates every request; `@PreAuthorize` enforces roles. The service layer runs business logic inside DB transactions. Three stores sit behind it: **MySQL/TiDB** (accounts, ledger, transactions — source of truth), **Redis** (OTPs, idempotency keys, TTL-based), and **Kafka** (async event stream fed by a transactional outbox). (Full diagram in Section 6.)

4. Tech Choices (and the reasoning — this is what they probe) Spring Boot 3 / Java 17 for the API; Spring Security + JWT for stateless, role-based auth; JPA/Hibernate with a `@Version` column for optimistic locking; TiDB (MySQL-compatible) for a scalable source of truth; Redis for fast auto-expiring keys; Kafka for durable, ordered, replayable events; Twilio for SMS; Maven to build; React 18 + Vite + Axios on the front. Two signature decisions: **money as Long subunits** (exact arithmetic, no IEEE-754 errors) and **optimistic over pessimistic locking** (no throughput collapse under concurrency).

5. Challenges Faced (the heart of the interview) - Double-charge on retry → idempotency keys in Redis; a duplicate key returns the cached result, DB never runs twice. - **Lost-update race on concurrent transfers** → optimistic locking; the second writer detects a stale version and gets a 409. - **Notification failure rolling back a payment** → Kafka transactional outbox; the event is written in the same DB transaction as the payment, and a background poller publishes it asynchronously. - **Crash mid-transfer / stuck transactions** → PENDING-first pattern + a scheduled ReconciliationService that auto-fails anything stuck over 5 minutes. - **Cross-currency correctness** → integer-safe FX conversion, with reversals careful to refund the converted amount, not the raw amount.

6. Results & Impact (be honest — it's a personal project) It's deployed and fully working end-to-end — you can register, get an account opened, deposit, transfer, trigger an OTP gate, reverse a transaction, and watch it all in the admin analytics. More importantly, it *correctly* handles the production scenarios most projects ignore: it provably rejects double-charges and concurrent-update races, and payments stay fast and atomic regardless of whether SMS or Kafka is healthy. The real impact is demonstrative: it shows I can design for correctness and failure, not just the happy path.

△ Honesty note: if asked for metrics (users, TPS), say plainly: *"It's a personal project, so I don't have production traffic numbers — but it's architected to scale horizontally because the API is stateless, and I can walk you through where the bottlenecks would be."* Never invent metrics.

7. Future Improvements (shows maturity — always have these ready) - Replace hardcoded FX rates with a live feed (Open Exchange Rates / ECB). - Make idempotency DB-backed (unique constraint) with Redis as a cache, so it survives a Redis outage. - Swap outbox polling for Change Data Capture (Debezium) to cut latency. - Add integration tests with Testcontainers and a real concurrency test. - Add observability — metrics, tracing, structured logs (Micrometer + Prometheus/Grafana).

□ LIVE DEMO GOTCHAS & TALKING POINTS

If you demo this live (or the interviewer opens the logs), be ready for these. Each is a chance to look sharp instead of caught off guard.

Login credentials (exact — case-sensitive):

Portal	Username	Password
Admin	admin	Admin@1234
Backoffice	backoffice1	Backoffice@1234

Common slip: it's `backoffice1` with the digit, not `backoffice`. OTP in dev mode is always `123456`.

The Kafka log message (now a clean warning, not a crash): On the free-tier deploy there's no Kafka broker, so I disabled the relay (`KAFKA_ENABLED=false`). The `OutboxPoller` now skips cleanly. If asked why Kafka isn't live:

"Kafka isn't provisioned on the free tier, so I gated the relay behind a flag. The key point is the payment path doesn't depend on it — transaction events still persist durably in the outbox table, and they'd relay automatically the moment a broker comes online. That's the whole value of the outbox pattern: the broker can be absent or down and payments are completely unaffected."

Cold start: first request after idle takes ~15–30s (Render free tier spinning up). Open the tabs a minute before the interview so it's warm.

The harmless startup warnings: `Spring Data Redis - Could not safely identify store assignment...` is classpath noise from having both Spring Data JPA and Redis present — your repos are correctly JPA (it logs "Found 0 Redis repository interfaces"). Cosmetic, not an error.

1. The 30-Second Pitch (memorize this cold)

"PayLedger is a production-style payment system I built from scratch — Spring Boot backend, three separate React portals, deployed live. It does double-entry accounting, multi-currency transfers with FX conversion, OTP-gated high-value payments, and role-based access for customers, back-office staff, and admins. The interesting part isn't the CRUD — it's that I deliberately solved the hard problems real payment systems hit: double-charges on retries, race conditions on concurrent transfers, and notification failures rolling back payments. I used idempotency keys, optimistic locking, and a Kafka transactional outbox to handle each one."

If they only remember one sentence about you, it should be: *"I didn't stop at CRUD — I solved the production problems."*

1B. The Full "Explain Your Project & What Technologies You Used" Answer (2–3 min)

This is the single most common interview opener. Deliver it in this order: **what it is** → **what it does** → **the architecture** → **the tech stack and why each piece** → **what makes it special**. Say it like this:

"PayLedger is a full-stack payment and ledger system I built from scratch — basically a simplified version of the infrastructure behind a digital bank or a payment gateway like Razorpay. It lets customers hold accounts, send money to each other, and handle deposits and withdrawals, with everything recorded in a proper double-entry ledger so the books always balance.

Architecturally it's a Spring Boot REST API on the backend, three separate React frontends for the three user roles — customer, back-office staff, and admin — and three backing data stores: MySQL for the source of truth, Redis for short-lived data, and Kafka for event streaming. It's all deployed live: the API on Render, the three frontends on Vercel.

For the **backend**, I used **Spring Boot 3** with **Java 17**. Authentication is **Spring Security with JWT** for stateless, role-based auth. For data access I used **JPA / Hibernate**, with a `@Version` column for optimistic locking. The database is **TiDB Cloud**, which is MySQL-compatible. I used **Redis** — hosted on Upstash — to store OTP codes and idempotency keys with automatic TTL expiry. **Apache Kafka** carries transaction events out of the system through a transactional outbox. **Twilio** sends the SMS notifications and OTPs. The build tool is **Maven**.

On the **frontend**, all three portals are **React 18** built with **Vite**, and they use **Axios interceptors** to automatically attach the JWT to every request and redirect to login on a 401.

What makes it more than a CRUD app is that I deliberately solved real payment-system problems: **idempotency keys** so a retried request never double-charges, **optimistic locking** so two concurrent

transfers can't corrupt a balance, and a **Kafka transactional outbox** so a notification failure can never roll back a payment. I also store all money as integer subunits — cents and paise — instead of decimals, so the arithmetic is always exact."

After that, pause and say: *"I can go deeper on any of those — the locking, the outbox pattern, or the double-entry model."* That hands them the next question on your terms.

Tech stack at a glance (so you can rattle it off)

Layer	Technology	Why you chose it (one line)
Language	Java 17	Industry standard for backend / fintech
Framework	Spring Boot 3	Fastest way to a production-grade REST API
Security	Spring Security + JWT	Stateless, scalable, role-enforced auth
ORM	JPA / Hibernate + @Version	Object mapping + optimistic locking
Database	TiDB Cloud (MySQL-compatible)	Source of truth; horizontally scalable
Cache / OTP	Redis (Upstash)	Fast, TTL-based — auto-expires OTPs & idempotency keys
Messaging	Apache Kafka	Durable, ordered, replayable event stream
SMS	Twilio API	OTP + transaction notifications
Build	Maven	Standard Java build & dependency management
Frontend	React 18 + Vite	Component model + sub-second builds
HTTP client	Axios (interceptors)	Auto-attach JWT, handle 401
Hosting	Render (API) + Vercel (×3 frontends)	Free-tier, CI-deployed

2. The Big Picture (understand this before anything else)

What the system does in one line: money moves between bank accounts, every movement is recorded as a balanced double-entry ledger, and three different user types interact with it through their own apps.

The three portals (and why they're separate):

1. **Customer portal** — register, log in, see balance, send money, raise support tickets.
2. **Backoffice portal** — staff open accounts for customers, deposit/withdraw cash, freeze accounts, reverse transactions, resolve tickets.
3. **Admin portal** — analytics dashboard, user management, system health monitoring.

They are **three completely separate React apps**, each with its own deployment. The customer app literally has no code that knows admin endpoints exist. This is security at the network level, not just hiding a button.

The data stores (and what each one is for):

- **TiDB Cloud (MySQL-compatible)** — the source of truth. Accounts, users, transactions, ledger entries, outbox events all live here.
- **Redis (Upstash)** — short-lived data: OTP codes (5-min expiry) and idempotency keys (24-hour expiry). Redis auto-deletes them on TTL, so no cleanup job needed.

- **Kafka** — the message pipeline that carries "a transaction happened" events out of the system asynchronously.

The golden rule of the whole system: money is **never** stored as a decimal. Everything is a **Long** integer in the smallest unit — paise for INR, cents for USD. **\$10.00** is stored as **1000**. (Reason explained in Q&A — this is a guaranteed question.)

3. The 10-Minute Spoken Walkthrough Script

This is structured so each section is ~1.5–2 minutes. Speak it like you're telling a story, not reading a list.

Part 1 — Opening (1 min)

"So the project is called PayLedger. It's a payment and ledger system — think of a simplified version of what runs behind a bank or a Razorpay. I built the full stack: a Spring Boot REST API, three React frontends for three different user roles, and it's deployed live on Render and Vercel.

The reason I built it this way is that most projects you see are basic CRUD apps. I specifically wanted to solve the problems that show up only in real money-handling systems — things like what happens when a payment request is sent twice, or when two transfers hit the same account at the same moment. So the architecture is built around correctness and reliability."

Part 2 — The core flow: how a transfer works (2 min)

"Let me walk through what happens when a customer sends money, because that touches most of the system.

The request comes into the **TransactionController**, which passes it to **TransactionService**. Inside one database transaction, it does this:

First, it checks an **idempotency key** in Redis. The frontend generates a unique key per payment attempt. If that key already exists, it means this is a duplicate — maybe the network retried — so I return the original result instead of charging again.

Second, it validates both accounts exist and are active. If the amount is above a high-value threshold — I set it at 10,000 rupees — it requires an **OTP**, which gets verified against Redis.

Third, it checks the source has enough balance.

Then it does the actual money movement using **double-entry accounting**: it debits the sender, writes a DEBIT ledger entry, credits the receiver, and writes a CREDIT ledger entry. Every transaction is always exactly one debit and one credit, so the books always balance.

If the currencies differ — say INR to USD — it runs an **FX conversion** on the credit side so the receiver gets the right amount in their own currency.

Finally it marks the transaction COMPLETED, saves the idempotency key, and writes an event to the outbox table — and I'll come back to why the outbox matters."

Part 3 — The hard problems I solved (3 min)

"The three things I'm most proud of are how I handle failure and concurrency.

Idempotency — In payments, the worst bug is double-charging. Networks retry. Users double-click. So every payment carries a unique key, stored in Redis with a 24-hour expiry. If the same key comes back, the database never runs the transaction twice — I just return the cached result. Same response, no double charge.

Optimistic locking — Imagine two transfers hitting the same account at the same instant. Both read a balance of 1,000. Both subtract. One overwrites the other — that's the 'lost update' problem, and in money it's catastrophic. I solved it with a `@Version` column on the account. Hibernate checks the version number at write time. If someone changed it since I read it, the write fails and the client gets a 409. I chose *optimistic* over pessimistic row-locking because 99% of transfers are between *different* accounts with no conflict — so I don't pay the cost of locking unless there's an actual race.

Kafka transactional outbox — This is the one I'd highlight most. When a payment completes, I want to send an SMS notification. The naive way is to call Twilio directly inside the transaction. But that couples two systems: if Twilio is slow, my payment is slow; if Twilio throws, does my payment roll back? That's wrong. So instead, inside the same database transaction as the payment, I write an event row to an 'outbox' table. Because it's the same transaction, the event is saved atomically with the payment — they either both commit or both roll back. Then a separate background poller picks up pending events every 10 seconds and publishes them to Kafka. The SMS becomes eventual but guaranteed, and payment speed is never affected by an external service."

Part 4 — Reliability and self-healing (1.5 min)

"A couple more reliability pieces.

I use a **PENDING-first pattern** — I save the transaction as PENDING before touching balances, so if the app crashes mid-transfer there's a record. But that means stuck PENDING rows can pile up. So there's a **ReconciliationService** — a scheduled job that runs every minute and marks any transaction stuck in PENDING for over 5 minutes as FAILED. That's automatic crash recovery. And because the client still has the idempotency key, they can safely retry.

There's also a **SystemBootstrap** that runs on every startup. It re-seeds the admin and backoffice credentials and the system cash account. So even if someone corrupts the admin password in the database, the system self-heals on the next restart — I can never get locked out of my own deployment."

Part 5 — Security and close (1.5 min)

"On security: authentication is stateless **JWT**. A `JwtFilter` runs on every request, validates the token, and loads the user's role into Spring Security's context. Then `@PreAuthorize` annotations enforce role-based access — a customer physically cannot call an admin endpoint. Passwords are BCrypt hashed. CORS is locked to exactly the three frontend URLs — no wildcard. And registration and high-value transfers are gated by OTP.

So overall: it's a system designed around the idea that money handling has to be correct, idempotent, and resilient to partial failures — and I built each of those guarantees explicitly rather than assuming the happy path. Happy to go deeper on any part."

4. Service-by-Service Reference (so you can answer "what does X do?")

Service / Component	What it does	One-line "why"
TransactionController	REST endpoints for transfer, deposit, withdraw, reverse, history	Thin layer — just routes to the service
TransactionService	The brain — runs the whole transfer flow inside one DB transaction	Where idempotency, OTP, double-entry, FX, and outbox all come together
AccountService	Create accounts, freeze/close, find active account, generate account numbers	Owns account lifecycle; holds the <code>SYSTEM_CASH</code> constant
IdempotencyKeyService	Stores/looks up idempotency keys in Redis (24h TTL)	Single responsibility — all idempotency Redis logic in one place
OtpService	Generate, store (5-min TTL), verify-and-consume OTPs via Redis	One-time-use codes; uses <code>SecureRandom</code> , not <code>Math.random()</code>
OutboxPoller	Scheduled job: reads PENDING outbox rows → publishes to Kafka → marks PUBLISHED	Decouples payment from external messaging
ReconciliationService	Scheduled job: marks PENDING transactions older than 5 min as FAILED	Safety net for the PENDING-first pattern; crash recovery
KafkaConsumerService	Consumes the transaction events from Kafka	The "downstream" side of the outbox pipeline
WebhookNotificationService	Sends notifications based on events	Eventual side-effects, off the payment path
SmsService / TwilioSmsService	Sends SMS via Twilio (OTP + transaction alerts)	Best-effort — wrapped in try/catch so it never breaks a payment
AnalyticsService	Aggregates volume, top senders/receivers; FX-normalizes to INR	Uses JOIN FETCH to avoid N+1 queries
SupportTicketService	Raise/resolve support tickets	The product-completeness feature
CurrencyConverter (util)	Converts subunits between 10 currencies using INR as the base rate	Pure integer-safe FX math
JwtFilter / JwtUtil	Validate JWT on every request, extract user + role	Stateless auth, no server-side sessions
SystemBootstrap	On startup: seed SYSTEM_CASH + admin/backoffice users, self-heal passwords	Can never get locked out
GlobalExceptionHandler	Maps exceptions to consistent JSON error responses	Clean, uniform error shape for the frontend

The key entities (data model)

- **User** — login identity + role (CUSTOMER / BACKOFFICE / ADMIN).

- **Account** — a bank account: number, balance (Long subunits), currency, status, and a **@Version** for optimistic locking.
- **Transaction** — a record of money moving: reference ID, source, destination, amount, type (TRANSFER/DEPOSIT/WITHDRAWAL/REVERSAL), status (PENDING/COMPLETED/FAILED/REVERSED).
- **LedgerEntry** — one side of a double-entry: DEBIT or CREDIT, linked to a transaction and an account.
- **OutboxEvent** — an event waiting to be published to Kafka (status PENDING/PUBLISHED).
- **IdempotencyKey** — (entity exists, but the live path uses Redis for speed).

The **SYSTEM_CASH** concept (worth understanding — it's clever and they may ask)

There's one special account called **SYSTEM_CASH** — the bank's cash float. Money can only **enter** the system via a deposit (debit SYSTEM_CASH, credit customer) and **exit** via a withdrawal (debit customer, credit SYSTEM_CASH). It's allowed to go negative because it represents the bank's clearing account. The invariant: **the sum of all ledger balances is always zero** — that's what proves the books are balanced. Customers can't transfer to/from it directly; only the deposit/withdraw endpoints touch it.

5. Interview Q&A — Drill These

Organized from most-likely to deep-dive. For each: the short answer first (say this), then the backup detail (if they push).

A. The guaranteed questions

Q: Why do you store money as Long (cents/paise) instead of Double or BigDecimal? Short: "Because floating-point can't represent money exactly. $0.1 + 0.2$ is 0.30000000000000004 in IEEE 754. In a payment system that's a real bug. I store every amount as an integer in the smallest unit — paise or cents — so all arithmetic stays exact and deterministic." If pushed on BigDecimal: "BigDecimal also works and is common in real systems, but it's slower and heavier for every operation. For this system, integer subunits give me exactness with simple, fast Long math, and the DB stays clean. The trade-off is I have to format for display, which I do in one place."

Q: What is idempotency and how did you implement it? Short: "Idempotency means making the same request twice has the same effect as making it once — critical for payments because networks retry. The client sends a unique key per payment. I store it in Redis mapped to the transaction's reference ID. On a duplicate key, I skip the whole flow and return the original result. No double charge." If pushed: "TTL is 24 hours. I check it as step one, before any balance changes, and I save it only after the transaction completes successfully."

Q: Explain optimistic locking. Why not pessimistic? Short: "I put a **@Version** column on the Account. Hibernate increments it on every update and checks it at write time. If two threads both read version 5 and both try to write, the second one sees the version already changed and the write fails with an optimistic lock exception — the client gets a 409. This prevents the lost-update problem." Why optimistic: "Pessimistic locking takes a DB row lock and serializes every writer to that account — throughput collapses under concurrency and you risk deadlocks. Optimistic assumes no conflict, which is true for the vast majority of transfers since they're between *different* accounts. You only pay a cost when an actual race happens, and then you can retry."

Q: What is the transactional outbox pattern and why use it? Short: "The problem: I need to publish an event / send a notification when a payment happens, but I can't safely do an external call inside my DB transaction. So instead I write an event row to an 'outbox' table *in the same transaction* as the payment. They commit together atomically. A separate background poller reads pending events and publishes them to Kafka, then marks them published." Why it matters: "It guarantees the event is never lost even if Kafka is down, and it

never makes the payment slow or roll back because of an external system. It's at-least-once delivery — eventual but guaranteed."

B. Architecture & design

Q: Why three separate frontends instead of one app with role-based routing? "Role separation at the network level, not just the UI level. The customer bundle contains zero knowledge of admin endpoints — there's literally nothing to inspect or exploit in the shipped JavaScript. Each portal has its own deployment and CORS policy. It mirrors how real fintech companies isolate internal tooling from customer-facing apps."

Q: Walk me through your authentication. "Stateless JWT. On login, I issue a signed token containing the username and role, 24-hour expiry. On every subsequent request, a `JwtFilter` extends `OncePerRequestFilter`, reads the Bearer token, validates the signature and expiry, and loads the user into Spring Security's context. Then `@PreAuthorize` on the endpoints enforces roles. No server-side session store, so it scales horizontally — any instance can serve any request."

Q: How does double-entry accounting work here? "Every transaction produces exactly two ledger entries: a DEBIT from the source and a CREDIT to the destination, for the same value. The sum across all accounts is always zero. This makes the system auditable — I can reconstruct any account's balance purely from its ledger entries, and any imbalance immediately signals a bug."

Q: How do you handle cross-currency transfers? "The debit happens in the source currency. On the credit side, I run `CurrencyConverter.convert()`, which converts via INR as the base. So an INR → USD transfer debits rupees from the sender and credits the FX-converted dollar amount to the receiver. Each ledger entry records its own currency. For reversals I'm careful to deduct the *converted* amount from the destination, not the raw source amount — otherwise cross-currency reversals would corrupt the balance."

C. Deeper / trade-off questions (these separate good candidates)

Q: Your outbox poller runs every 10 seconds. Isn't that slow / what about scale? "For this project's scale it's fine — SMS notifications being a few seconds late is acceptable. At higher scale I'd reduce the interval, batch publishes, and add a `LIMIT` so one poll doesn't pull thousands of rows. The cleaner production approach is Change Data Capture — something like Debezium tailing the database log and streaming outbox inserts to Kafka in near-real-time, removing polling entirely."

Q: What happens if the app crashes right after debiting the source but before crediting the destination? "It can't leave money lost, because the debit, credit, and both ledger entries are all in one `@Transactional` method — if it doesn't commit, the database rolls all of it back. The transaction row itself is PENDING until the final COMPLETED save. If the JVM dies mid-flight, the DB transaction never commits, so balances are untouched, and the PENDING row gets cleaned up by `ReconciliationService`. The client retries safely with the idempotency key."

Q: Your idempotency keys are in Redis. What if Redis goes down? "Honest answer: that's the weakest link in the current design. If Redis is unavailable, idempotency protection is lost for that window. In production I'd back idempotency with a unique constraint in the database as the durable source of truth and use Redis only as a fast cache in front of it. There's actually an `IdempotencyKey` entity in the codebase for exactly that evolution."

Q: How do you prevent N+1 queries in analytics? "The analytics aggregation uses JOIN FETCH so related entities load in a single query instead of one query per row. I also paginate transaction history at the database level — earlier I was fetching everything and merging in memory, which doesn't scale; now the DB does the pagination and the app only receives one page."

Q: Why Kafka and not just an async method or a queue table? "Honest framing: for this scale, Kafka is arguably overkill and a simple async approach would work. I used it deliberately to learn the outbox-to-broker

pattern that real event-driven systems use. Kafka gives durable, ordered, replayable events and lets multiple independent consumers react to the same payment event — that's the architecture I wanted to demonstrate. I key events by transaction reference ID so all events for one transaction land on the same partition and stay ordered."

D. "Gotcha" / honesty questions — answer with confidence, not defensiveness

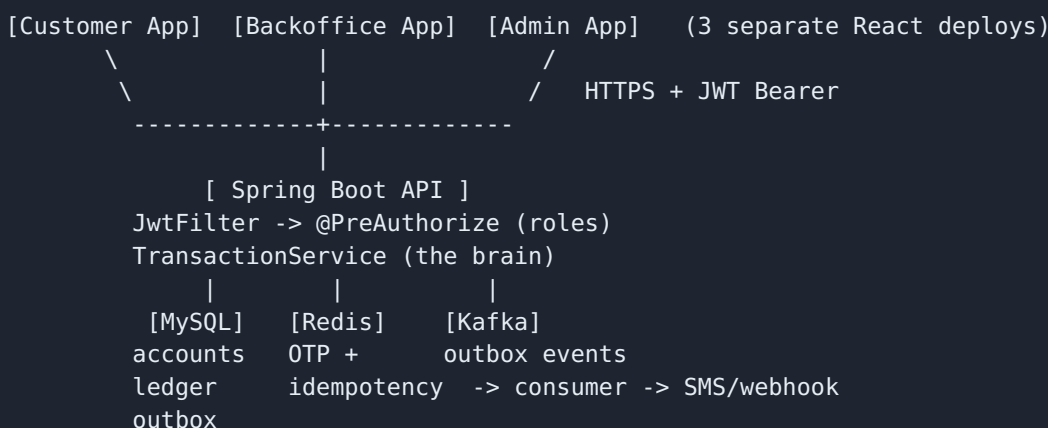
Q: What are the weaknesses or what would you do differently? Pick 2–3 and own them — this is a strength signal: - "FX rates are hardcoded — in production I'd pull a live feed like Open Exchange Rates." - "Idempotency should be DB-backed for durability, not Redis-only." - "I'd replace the outbox polling with CDC (Debezium) to remove the latency." - "OTP has a dev-mode bypass (always 123456) for the demo — in production that's off and uses SecureRandom + real Twilio."

Q: How would you test this? "There's a unit test for TransactionService using Mockito to mock the repositories — testing the balance math, insufficient-balance rejection, and the idempotency replay path. To go further I'd add integration tests with Testcontainers spinning up real MySQL and Redis, and a concurrency test that fires two simultaneous transfers at one account to prove the optimistic lock actually rejects the second writer."

Q: How does it scale to thousands of transactions per second? "The stateless JWT design means I can run many API instances behind a load balancer. The bottleneck would be the database and contention on hot accounts. I'd add read replicas for queries/analytics, partition or shard high-traffic accounts, move idempotency to a DB constraint, and replace outbox polling with CDC. The double-entry model itself stays correct regardless of scale."

Q: Reconciliation marks stuck transactions FAILED — but what if it was actually mid-commit? "The 5-minute cutoff is deliberately very conservative — normal transactions finish in milliseconds. A transaction still PENDING after 5 minutes is genuinely stuck, not slow. And since the actual money movement is atomic within the DB transaction, a 'failed' reconciliation just marks an attempt that never committed; balances were never changed. The client can retry with the same idempotency key."

6. Whiteboard Cheat (if they ask you to draw it)



Draw the transfer flow as numbered steps: 1. Idempotency check (Redis) 2. Validate accounts + OTP if high-value 3. Balance check 4. Debit source + DEBIT ledger entry 5. Credit destination (FX if needed) + CREDIT ledger entry 6. Mark COMPLETED 7. Write outbox event (same transaction!) 8. Poller → Kafka → SMS (async, later)

7. Final Tips for the Room

- **Lead with the problem, then the solution.** "Networks retry, so I needed idempotency" lands better than "I used Redis."
- **Use the words they want to hear** naturally: *idempotency, optimistic locking, double-entry, atomicity, eventual consistency, transactional outbox, stateless auth, at-least-once delivery.*
- **When you don't know, say so and reason out loud.** "I haven't measured that, but I'd expect the bottleneck to be DB contention because..." — reasoning beats bluffing every time.
- **Have one number ready:** high-value OTP threshold = 10,000 INR; 10 supported currencies; JWT expiry 24h; reconciliation cutoff 5 min. Specifics signal you actually built it.
- **Close every deep answer by inviting the next question:** "...happy to go deeper on the locking if useful." It shows command of the material.
- **Own the weaknesses proactively.** Saying "Redis-only idempotency is the weak link, here's how I'd fix it" makes you look senior. Hiding it makes you look junior when they find it.